Students:
Nguyen Huu Truong Giang [u6110634@udg.edu]

Lab 2: Resolved-rate motion control

1 Introduction

Kinematic control in robotics is a common approach used to solve the problem of controlling a robotic arm to move its end effector to a desired goal position or to track a moving point with the end effector. To solve this problem, multiple methods can be applied, such as the inverse Jacobian method, Jacobian transpose method, damped least squares method, and feedforward control.

In this lab, we conduct experiments using a 2D planar robotic arm to control the end effector toward a goal position using different methods: **the Pseudoinverse Jacobian method**, **Jacobian Transpose method**, and **damped least squares method**. We then compare the error and the time required to reach the desired position for each method.

Section 2 presents the methodology used to calculate the robot kinematics and the equations for the different control methods. Section 3 visualizes the results. Section 4 answers the discussion questions. Section 5 concludes the lab.

2 Methodology

The implementation was carried out in Python using the `NumPy` library for numerical computations and `Matplotlib` for visualizing the robot motion and results. The robot kinematics, Jacobian matrix equations and control algorithms, along with their code implementations, are described below.

2.1 Robot Kinematics

The Denavit–Hartenberg (DH) convention is used to construct the homogeneous transformation matrix from the base frame to each joint of the robotic arm. The transformation between frame $n-1$ and frame n is given by:

$$T_n^{n-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) & 0 & 0 \\ \sin(\theta_i) & \cos(\theta_i) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\alpha_i) & -\sin(\alpha_i) & 0 \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where:

- d_i is the displacement along the Z-axis,

- θ_i is the rotation about the Z-axis,
- a_i is the displacement along the X-axis,
- α_i is the rotation about the X-axis.

Figure 1 shows the implementation of the DH transformation in Python.

```python
def DH(d, theta, a, alpha):
    # 1. Build matrices representing elementary transformations (based on input parameters).
    T_d = np.array([[1, 0, 0, 0],
                    [0, 1, 0, 0],
                    [0, 0, 1, d],
                    [0, 0, 0, 1]])
    T_theta = np.array(
        [
            [cos(theta), -sin(theta), 0, 0],
            [sin(theta), cos(theta), 0, 0],
            [0, 0, 1, 0],
            [0, 0, 0, 1],
        ]
    )
    T_a = np.array([[1, 0, 0, a],
                    [0, 1, 0, 0],
                    [0, 0, 1, 0],
                    [0, 0, 0, 1]])
    T_alpha = np.array(
        [
            [1, 0, 0, 0],
            [0, cos(alpha), -sin(alpha), 0],
            [0, sin(alpha), cos(alpha), 0],
            [0, 0, 0, 1],
        ]
    )
    # 2. Multiply matrices in the correct order (result in T).
    T = T_d @ T_theta @ T_a @ T_alpha
    return T
```

Figure 1: Implementation code for the DH transformation matrix

After obtaining the individual transformation matrices, the overall kinematic chain from the base frame to joint n is computed as:

$$T_n^0 = T_1^0 T_2^1 T_3^2 \cdots T_n^{n-1} = \begin{bmatrix} R_n^0(q) & O_n^0(q) \\ 0 & 0 \end{bmatrix}$$

This multiplication yields the final homogeneous transformation matrix describing the position and orientation of joint n with respect to the base frame.

Figure 2 shows the implementation of the kinematic chain computation.

```python
def kinematics(d, theta, a, alpha):
    T = [np.eye(4)]  # Base transformation
    # For each set of DH parameters:
    for i in range(len(d)):
        # 1. Compute the DH transformation matrix.
        n_1_T_n = DH(d[i], theta[i], a[i], alpha[i])
        # 2. Compute the resulting accumulated transformation from the base frame.
        T_0_n = T[-1] @ n_1_T_n
        # 3. Append the computed transformation to T.
        T.append(T_0_n)
    return T
```

Figure 2: Implementation code for calculating the kinematic chain

2.2 Jacobian Matrix

After obtaining the kinematic chain of the robotic arm, the geometric Jacobian matrix is computed to map joint velocities to the end-effector velocity. The equations used to construct the Jacobian matrix are described below, along with the corresponding code implementation shown in the figure.

$$J_n^0 = \begin{bmatrix} J_1 & J_2 & J_3 & \cdots & J_n \end{bmatrix}$$

For each column:

Revolute joint:

$$J_i = \begin{bmatrix} \mathbf{z}_{i-1} \times (\mathbf{O}_n - \mathbf{O}_{i-1}) \\ \mathbf{z}_{i-1} \end{bmatrix}$$

Prismatic joint:

$$J_i = \begin{bmatrix} \mathbf{z}_{i-1} \\ \mathbf{0} \end{bmatrix}$$

where:

- $\mathbf{z}_{i-1}$ is the z-axis of joint $i-1$,
- $\mathbf{O}_k$ represents the position vector extracted from the corresponding homogeneous transformation matrix.

```python
def jacobian(T, revolute):
    # 1. Initialize J and O.
    n_dof = len(revolute)
    J = np.zeros((6, n_dof))
    O = T[-1][0:3, 3]
    # 2. For each joint of the robot
    for i in range(n_dof):
        #   a. Extract z and o.
        i_1_T_i = T[i]
        z = i_1_T_i[0:3, 2]
        o = i_1_T_i[0:3, 3]
        #   b. Check joint type.
        if revolute[i] == True:
            cross_vector = np.array([np.cross(z, (O - o))]).T
            z = np.array([z]).T
            J_i = np.vstack((cross_vector, z))
        else:
            J_i = np.vstack((z, np.zeros((3, 1))))
        #   c. Modify corresponding column of J.
        J[:, i] = J_i[:, 0]
    return J
```

Figure 3: Implementation code for computing the Jacobian matrix

2.3 Inverse and Pseudoinverse (Moore–Penrose)

There are multiple methods to map joint velocities to end-effector velocities. One approach is to use the inverse of the Jacobian matrix when it is square and full-rank, or the pseudoinverse when the Jacobian is non-square. The corresponding equations and code implementations are presented below.

Inverse Jacobian:

$$\dot{\zeta} = J^{-1}(\mathbf{q})\,\dot{\mathbf{x}}_E \quad \text{(square and full-rank Jacobian)}$$

Pseudoinverse Jacobian:

$$\dot{\zeta} = J^{\dagger}(\mathbf{q})\,\dot{\mathbf{x}}_E \quad \text{(square or non-square full-rank Jacobian)}$$

The code implementation of the pseudoinverse Jacobian is shown in Fig. 4.

```python
# Calculate joint velocity

dq = np.linalg.pinv(J) @ (K @ err)  # Control solution using Pseudoinverse
```

Figure 4: Code implementation of the pseudoinverse Jacobian equation

2.4 Jacobian transpose (no linearisation)

We have another method that help us do not need to do the inverse or pseudoinverse Jacobian which may reduce the time for computation. The equation and the code for implementation is provided below:

$$\dot{\zeta} = J^T(\mathbf{q})\,\dot{\mathbf{x}}_E \quad \text{(No need for inverse matrix)}$$

The code implementation is shown in Fig:

```
# Calculate joint velocity

dq = J.T @ (K @ err) # Control solution using Transpose
```

Figure 5: The code implementation for transpose Jacobian matrix

2.5 Damped least-squares (DLS)

Even though we use Inverse, Pseudoinverse or transpose Jacobian matrix we still deal with the singularity problem. To mitigate it, we introduce the Damped least-square equation (DLS) to add small value damping to ensure the matrix is full rank or rank deficient. The equation with code implementation is provided below:

$$\dot{\zeta} = J^T(\mathbf{q})\,(J(\mathbf{q})\,J^T(\mathbf{q}) + \lambda^2 I)^{-1}\,\dot{\mathbf{x}}_E$$

The code implementation is presented in Fig and Fig:

```
# Damped Least-Squares
def DLS(A, damping):
    """
    Function computes the damped least-squares (DLS) solution to the matrix inverse problem.

    Arguments:
    A (Numpy array): matrix to be inverted
    damping (double): damping factor

    Returns:
    (Numpy array): inversion of the input matrix
    """
    DLS_matrix = A.T @ np.linalg.inv(A @ A.T + damping**2 * np.identity(len(A)))
    return DLS_matrix  # Implement the formula to compute the DLS of matrix A.
```

Figure 6: Code implementation of computing DLS equation

```
# Calculate DLS
damping_factor = 0.1
DLS_matrix = DLS(J, damping_factor)

# Calculate joint velocity

dq = DLS_matrix @ (K @ err)  # Control solution using DLS
```

Figure 7: Code implementation of how mapping from end effector velocity to joint velocity

3 Results

Two main experiments are conducted in this study. In the first experiment, we record the trajectory of the robotic arm end effector and the joint positions over time under predefined joint velocities. In the second experiment, a resolved-rate motion control algorithm is applied to drive the robot arm toward a desired end-effector position using different control methods. The performance of these methods is evaluated by comparing the evolution of the error norm over time.

In this laboratory experiment, a 2-DOF planar robot with two revolute joints is used. The robot model, including its coordinate frames and Denavit–Hartenberg (DH) parameters, is illustrated in Fig. 8.

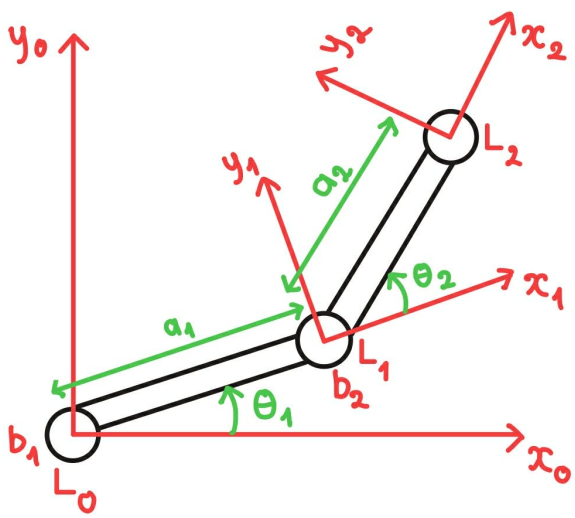

Figure 8: Visualization of the 2-DOF planar robot model

The robot configuration is initialized according to the DH parameters listed in Table 1.

DOF	θ_i	d_i	a_i	α_i	Home
1	q_1	0	$a_1 = 0.75$	0	0.2
2	q_2	0	$a_2 = 0.5$	0	0.5

Table 1: Denavit–Hartenberg parameters of the 2-DOF robot arm

3.1 Robot Kinematics

To conduct the first experiment, the joint velocities of the robot arm, the total simulation time, and the sampling interval are initialized. The parameters used in the simulation are summarized in Table 2.

Joint velocity	Sampling time	Total simulation time
$[0.5, 0.7]$	0.01	10

Table 2: Parameters for the robot kinematics experiment

After setting these parameters, the robot arm is simulated to move with the predefined joint velocities. The robot motion, end-effector trajectory, and joint positions over time are recorded. The visualization of the robot motion and end-effector path is shown in Fig. 9, while the evolution of joint positions over time is shown in Fig. 10.

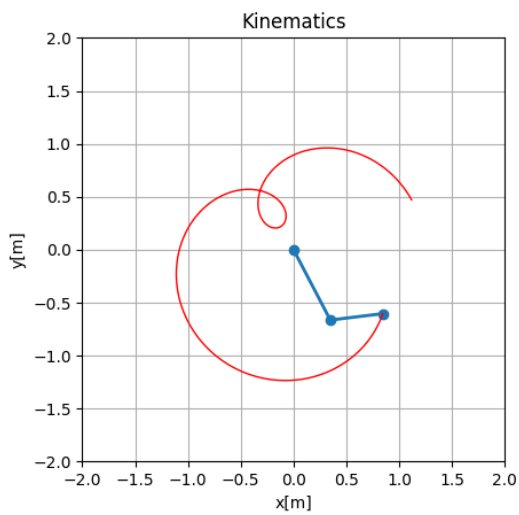

Figure 9: Simulation of the planar robot and the end-effector trajectory

The implementation code used for visualizing the robot motion and plotting the joint positions is shown in Fig. 11.

3.2 Resolved-Rate Motion Control Algorithm

To compare different control methods, the desired end-effector position, controller gains, and simulation parameters are first defined. These parameters are listed in Table 3.

Desired position	Gain	Sampling time	Total simulation time	Damping factor
$[0.0, 1.0]$	$[1.0, 1.0]$	$1/60$	10	0.2

Table 3: Parameters for comparing resolved-rate control methods

Using these parameters, simulations are performed with different resolved-rate motion control methods. The resulting end-effector trajectories are shown in Fig. 12. Additionally, the norm of the error between the current end-effector position and the desired position over time is shown in Fig. 13.

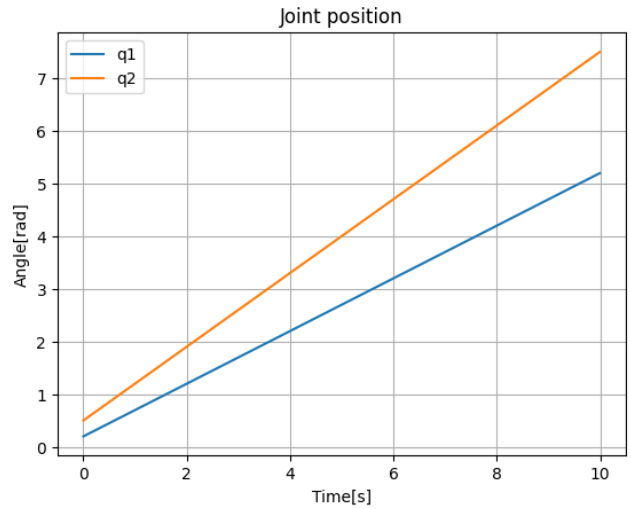

Figure 10: Joint positions of the robot during the simulation

```
# Run simulation
animation = anim.FuncAnimation(fig, simulate, tt,
                               interval=10, blit=True, init_func=init, repeat=False)

plt.show()

# Plot joint positions
t_rec = np.arange(len(q1_record)) * dt # Time vector for recorded joint angles
plt.plot(t_rec, q1_record, label='q1')
plt.plot(t_rec, q2_record, label='q2')
plt.legend()
plt.title('Joint position')
plt.xlabel('Time[s]')
plt.ylabel('Angle[rad]')
plt.grid()
plt.show()
```

Figure 11: Code implementation for visualizing robot motion and joint positions

As shown in Fig. 13, the Jacobian transpose method results in a significantly larger steady-state error compared to the pseudoinverse and damped least-squares methods. Both the pseudoinverse and damped least-squares approaches achieve lower error values that converge close to zero, indicating superior tracking performance.

The code implementation of the simulation function for controlling the robot arm using the resolved-rate motion control algorithm and the code for plotting the error norm of each method are shown in Fig. 14 and Fig. 15, respectively.

4 Discussion Questions

Question 1: What are the advantages and disadvantages of using kinematic control in robotic systems?

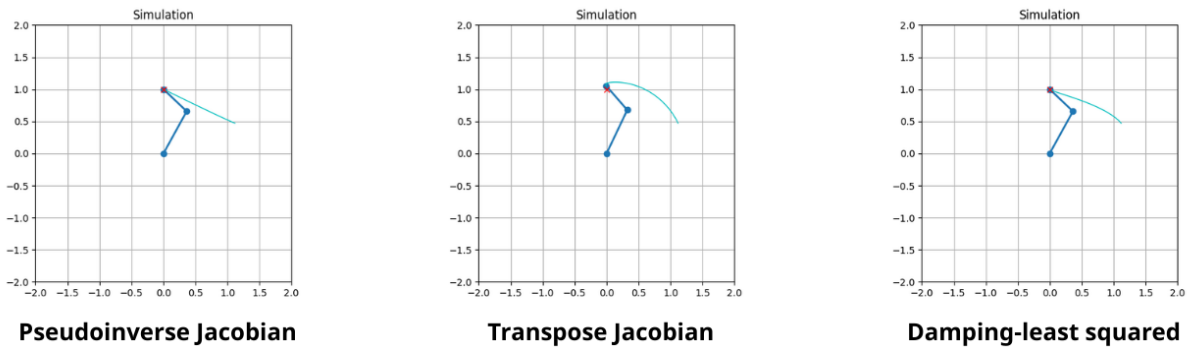

Figure 12: End-effector trajectories under different control methods

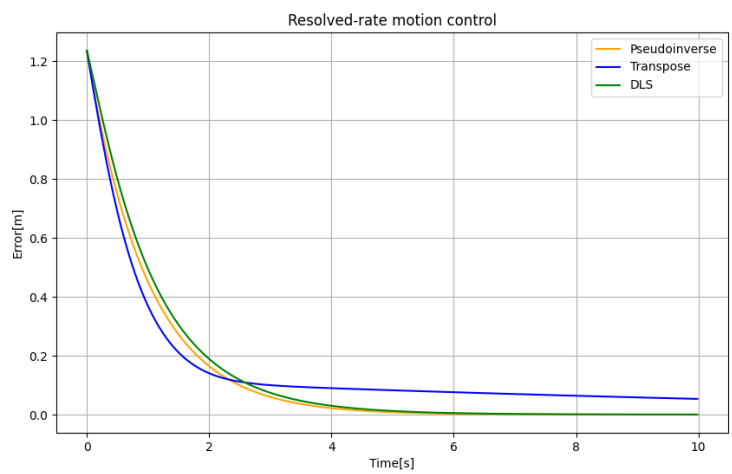

Figure 13: Evolution of the end-effector position error norm over time

Answer: Kinematic control in robotic systems offers several advantages. It is relatively simple and easy to implement, as it does not require knowledge of the robot's dynamics. Therefore, only the kinematic model and the robot configuration are needed to generate control commands. This makes kinematic control computationally efficient and suitable for low or mid speed control

However, kinematic control also has some disadvantages. Since dynamic effects such as mass, inertia, and external forces are not considered, the control performance degrades at higher speeds. This may lead to poor tracking accuracy or instability when applied to real robotic hardware. Additionally, kinematic control methods are sensitive to kinematic singularities, where the Jacobian matrix loses rank and the robot cannot generate velocities in certain directions.

Question 2: Give examples of control algorithms that may be used in the robot's

```python
# Simulation loop
def simulate(t):
    global d, q, a, alpha, revolute, sigma_d
    global PPx, PPy

    # Update robot
    T = kinematics(d, q, a, alpha)
    J = jacobian(T, revolute) # Implement!

    # Get Jacobian of only 2 first rows (for position control)
    J = J[0:2, :]

    # Update control
    sigma = T[-1][0:2, 3]        # Position of the end-effector
    err = np.array([sigma_d - sigma]).T        # Control error

    # Calculate DLS
    damping_factor = 0.2
    DLS_matrix = DLS(J, damping_factor)

    # Calculate joint velocity

    # dq = np.linalg.pinv(J) @ (K @ err)  # Control solution using Pseudoinverse

    # dq = J.T @ (K @ err) # Control solution using Transpose

    dq = DLS_matrix @ (K @ err)  # Control solution using DLS


    dq = np.array([dq[0][0], dq[1][0]]) # Get two joint velocities
    q += dt * dq

    # Record error
    error_record.append(np.linalg.norm(err))

    # Update drawing
    P = robotPoints2D(T)
    line.set_data(P[0,:], P[1,:])
    PPx.append(P[0,-1])
    PPy.append(P[1,-1])
    path.set_data(PPx, PPy)
    point.set_data([sigma_d[0]], [sigma_d[1]])

    return line, path, point
```

Figure 14: Code implementation for simulating resolved-rate motion control

hardware to follow the desired joint velocities generated by the resolved-rate motion control algorithm.

Answer: A commonly used control algorithm at the hardware level is the proportional–integral–derivative (PID) controller. PID control can be applied to each joint independently to ensure that the actual joint velocities accurately track the desired joint velocities produced by the resolved-rate motion control algorithm. Other possible approaches include proportional (P) control or proportional-integral (PI) control, depending on the complexity and accuracy requirements of the system

5 Conclusion

In conclusion, this laboratory exercise demonstrates the implementation of robotic kinematics and resolved-rate motion control. The kinematic chain and Jacobian matrix of a robotic arm are computed based on the robot configuration and used for kinematic control. Several resolved-rate control methods, including the Jacobian pseudoinverse, Jacobian transpose, and damped least-squares approaches, are implemented and compared.

```python
import matplotlib.pyplot as plt
import numpy as np

# Initialize error records for each method
error_record_transpose = []
error_record_Pseudoinverse = []
error_record_DLS = []

# Read error records from files data
with open('data/error_record_transpose.txt', 'r') as f:
    for line in f:
        error_record_transpose.append(float(line.strip()))

with open('data/error_record_Pseudoinverse.txt', 'r') as f:
    for line in f:
        error_record_Pseudoinverse.append(float(line.strip()))

with open('data/error_record_DLS.txt', 'r') as f:
    for line in f:
        error_record_DLS.append(float(line.strip()))

# Plotting the error records for each method
dt = 1.0/60.0 # Simulation time
time_Pseudoinverse = np.arange(len(error_record_Pseudoinverse)) * dt # Time params for plotting in x axis for Pseudoinverse
time_transpose = np.arange(len(error_record_transpose)) * dt # Time params for plotting in x axis for transpose
time_DLS = np.arange(len(error_record_DLS)) * dt # Time params for plotting in x axis for DLS
plt.figure(figsize=(10, 6))
plt.plot(time_Pseudoinverse, error_record_Pseudoinverse, label="Pseudoinverse", color = 'orange')
plt.plot(time_transpose, error_record_transpose, label="Transpose", color = 'blue')
plt.plot(time_DLS, error_record_DLS, label="DLS", color = 'green')
plt.title("Resolved-rate motion control")
plt.xlabel("Time[s]")
plt.ylabel("Error[m]")
plt.grid()
plt.legend()
plt.show()
```

Figure 15: Code implementation for plotting the error norm over time for different methods

Through simulation and analysis of the end-effector position error, the advantages and limitations of each control method are observed. The results show that the pseudoinverse and damped least-squares methods provide better tracking performance and improved stability compared to the Jacobian transpose method. Overall, this lab provides valuable insight into the behavior and practical considerations of kinematic control strategies in robotic systems.